

Compresión y Descompresión de Archivos de Texto mediante el Algoritmo de Huffman

Integrante	Código
Chanin Fernando Huaman Farfan	234893
Diego Abraham Tunque Mamani	234899
Cristhofer Antony Mora Mamani	241441

1. Estructura de la Solución

El proyecto está implementado en **C#** sobre **.NET 8**, organizado en una arquitectura modular de cuatro capas: modelos, núcleo algorítmico, entrada/salida y utilidades. Cada módulo tiene una responsabilidad única, lo que facilita la comprensión, el testing y la exposición del proyecto.

1.1 Organización de Archivos

Ruta	Descripción
HuffmanCompressor.csproj	Archivo de proyecto .NET 8
Program.cs	Punto de entrada, menú de consola y coordinación
Models/HuffmanNode.cs	Nodo del árbol de Huffman (hoja o interno)
Core/FrequencyAnalyzer.cs	Análisis de frecuencia de caracteres
Core/HuffmanTree.cs	Construcción del árbol con min-heap (PriorityQueue)
Core/CodeGenerator.cs	Generación de códigos binarios mediante DFS
Core/Compressor.cs	Codificación del texto a bytes comprimidos
Core/Decompressor.cs	Decodificación de bytes a texto original
IO/HuffFileWriter.cs	Serialización del archivo .huff al disco
IO/HuffFileReader.cs	Lectura y deserialización del archivo .huff
Utils/StatsDisplay.cs	Cálculo y presentación de estadísticas
Utils/TreeVisualizer.cs	Visualización del árbol de Huffman en consola

1.2 Flujo de Datos

Compresión:

Archivo .txt → FrequencyAnalyzer (frecuencias) → HuffmanTree (árbol) → CodeGenerator (tabla de códigos) → Compressor (bytes comprimidos) → HuffFileWriter → Archivo .huff

Descompresión:

Archivo .huff → HuffFileReader (frecuencias + bytes) → HuffmanTree (reconstrucción del árbol) → Decompressor (texto original) → Archivo .txt

1.3 Formato del Archivo .huff

El archivo comprimido es autocontenido: incluye toda la información necesaria para la descompresión sin archivos auxiliares.

Sección	Tamaño	Contenido
Cabecera	4 bytes	Número de entradas en la tabla (N)
Frecuencias	$N \times 6$ bytes	Cada entrada: 2 bytes (carácter) + 4 bytes (frecuencia)
Padding	1 byte	Bits de relleno al final del bloque de datos
Datos	Variable	Bits comprimidos empaquetados en bytes (MSB primero)

2. Principales Clases Implementadas

HuffmanNode — *Models/HuffmanNode.cs*

Representa un nodo del árbol binario de Huffman. Los nodos hoja almacenan un carácter y su frecuencia; los nodos internos combinan dos hijos y su frecuencia es la suma de ambos. La propiedad *IsLeaf* permite distinguir entre ambos tipos sin flags externos.

FrequencyAnalyzer — *Core/FrequencyAnalyzer.cs*

Recorre el texto una sola vez — $O(n)$ — y devuelve un diccionario con la frecuencia de cada carácter. Es el primer paso del pipeline de compresión.

HuffmanTree — *Core/HuffmanTree.cs*

Implementa la **estrategia voraz** del algoritmo de Huffman: usa un min-heap (*PriorityQueue*) para extraer repetidamente los dos nodos de menor frecuencia, combinarlos en un nodo padre, y reinsertarlo. Al terminar queda un único nodo: la raíz del árbol óptimo. Complejidad: $O(n \log n)$. Maneja el caso borde de un solo carácter distinto.

CodeGenerator — *Core/CodeGenerator.cs*

Recorre el árbol en profundidad (DFS): por cada nodo izquierdo agrega '0', por cada derecho agrega '1'. Al llegar a una hoja, asigna el código acumulado al carácter. El resultado es un *Dictionary<char, string>* que mapea cada carácter a su código binario de longitud variable.

Compressor — *Core/Compressor.cs*

Reemplaza cada carácter del texto por su código Huffman, genera un flujo de bits y lo empaqueta en bytes. Calcula el padding necesario para que el último byte quede completo.

Decompressor — *Core/Decompressor.cs*

Navega el árbol de Huffman bit a bit: cada '0' va a la izquierda, cada '1' a la derecha. Al llegar a una hoja, emite el carácter y vuelve a la raíz. Usa el padding para saber exactamente cuántos bits válidos hay en el último byte.

HuffFileWriter / HuffFileReader — *IO/*

Serializan y deserializan el archivo .huff usando *BinaryWriter* y *BinaryReader*. Escriben la cabecera (tabla de frecuencias), el byte de padding y los datos comprimidos en un formato binario determinístico.

TreeVisualizer — *Utils/TreeVisualizer.cs*

Muestra el árbol de Huffman como diagrama indentado en consola con conectores de línea (■■■ y ■■■■). Cada nodo muestra si es interno o hoja, su frecuencia, y el bit asignado (0 = izquierda, 1 = derecha). Filtra el nodo ficticio del caso de un solo carácter.

3. Resultados Obtenidos en las Pruebas

Se realizaron pruebas de compresión y descompresión con tres archivos de texto de diferente tamaño y contenido. En todos los casos, el archivo restaurado fue **idéntico byte a byte** al original, verificado mediante comparación directa.

3.1 Resumen Comparativo

Archivo	Tamaño original	Tamaño comprimido	Chars. únicos	Compresión
prueba_parrafos.txt	1.91 KB	1.32 KB	55	30.97%
prueba_repeticiones.txt	320 bytes	301 bytes	30	5.94%
prueba_simple.txt	70 bytes	152 bytes	22	-117.14%

3.2 Prueba 1: prueba_parrafos.txt

Contenido: Texto en español de 1.91 KB sobre algoritmos voraces y el algoritmo de Huffman. Representa un caso de uso realista con distribución natural de caracteres.

Resultado: Compresión del **30.97%**. La cabecera ocupa 335 bytes (55 caracteres únicos × 6 bytes + 5 bytes fijos) y los datos comprimidos 1018 bytes. Los caracteres más frecuentes (espacio, 'e', 'o', 'a') obtuvieron códigos de 3-4 bits, significativamente menos que los 8 bits de ASCII, lo que explica la compresión efectiva.

Carácter	Frecuencia	Código Huffman	Bits
(espacio)	296	110	3
e	190	001	3
o	179	1111	4
a	153	1011	4
n	125	1001	4

3.3 Prueba 2: prueba_repeticiones.txt

Contenido: Archivo de 320 bytes con alta repetición del carácter 'a' (113 apariciones) seguido de repeticiones decrecientes de 'b', 'c', 'd', etc.

Resultado: Compresión del **5.94%**. Aunque los datos comprimidos son pequeños (116 bytes), la cabecera de 185 bytes (30 caracteres únicos) limita el ahorro neto. El carácter 'a' obtuvo el código más corto posible: '11' (2 bits), confirmando la optimalidad del algoritmo voraz.

3.4 Prueba 3: prueba_simple.txt

Contenido: Una sola línea de 70 bytes con 22 caracteres distintos.

Resultado: Compresión **negativa (-117.14%)**. El archivo comprimido (152 bytes) es más grande que el original. Esto es un comportamiento esperado y bien documentado: la cabecera que almacena la tabla de frecuencias (137 bytes) supera ampliamente el ahorro obtenido por la codificación. Huffman requiere un volumen mínimo de datos para que la compresión sea rentable.

3.5 Verificación de Integridad

En los tres casos, tras descomprimir el archivo .huff se generó un archivo .restored.txt que fue comparado con el original. La verificación confirmó que los archivos son **idénticos byte a byte**, validando la correcta implementación del pipeline completo: análisis → construcción del árbol → codificación → empaquetado → deserialización → decodificación → reconstrucción.

3.6 Observaciones

- La compresión de Huffman es más efectiva en archivos de mayor tamaño con distribución desigual de caracteres.
- El overhead de la cabecera es fijo por carácter único (6 bytes), lo que penaliza archivos pequeños con muchos caracteres distintos.
- El carácter más frecuente siempre obtiene el código más corto, confirmando la propiedad voraz del algoritmo: decisiones localmente óptimas producen un resultado globalmente óptimo.
- No se utilizaron librerías externas para la implementación del algoritmo. Toda la lógica de Huffman fue desarrollada desde cero por el grupo.